# Sequence Diagram - Tạo Đơn Hàng & Kiểm Soát Tồn Kho Đồng Thời (Order Creation)

**Frontend (Next.js)**

**Backend (NestJS)**

Khách hàng (User)

Page: CheckoutPage (app/checkout/page.tsx)

Zustand: useCartStore (store/useCartStore.ts)

Client: order.service.ts (services/order.service.ts)

Guard: JwtAuthGuard (modules/auth/guards/jwt-auth.guard.ts)

Controller: OrdersController (modules/orders/orders.module.ts)

Service: OrdersService (modules/orders/orders.service.ts)

Service: PricingService (modules/pricing/pricing.service.ts)

Service: PromotionsService (modules/promotions/promotions.service.ts)

Database: PrismaService (prisma/prisma.service.ts)

1 Truy cập /checkout & nhấn "Tiếp tục thanh toán"

2 handleCreateOrder()

**opt** [Đã có đơn PENDING trước đó và giỏ hàng không thay đổi (pendingCartSignature === currentSignature)]

3 Điều hướng thẳng sang /checkout/payment?orderId=pendingOrderId (Tái sử dụng đơn)

**opt** [Đã có đơn PENDING trước đó nhưng giỏ hàng đã thay đổi]

4 cancelOrder(pendingOrderId)

5 POST /api/v1/orders/:id/cancel (Kèm JWT)

6 200 OK: Đã hủy đơn cũ

7 createOrder({ items, promotionCode })

8 POST /api/v1/orders
Headers: Authorization: Bearer <JWT>
Body: CreateOrderDto

**alt** [Token không hợp lệ / Hết hạn]

9 401 Unauthorized: { success: false, message: "Token không hợp lệ" }

10 Ném ApiError(401)

11 Điều hướng về /auth/login?next=/checkout

**[Token hợp lệ (Trích xuất req.user = { sub: userId })]**

12 Chuyển tiếp Request kèm req.user

13 create(userId, dto)

**alt** [Đơn hàng không có sản phẩm nào (dto.items.length === 0)]

14 throw BadRequestException('An order must contain at least one item.')

15 400 Bad Request

16 Hiển thị lỗi

17 calculate(pricingInput)

18 { subtotal, lines: [{ productId, unitPrice, lineTotal }] }

**opt** [Có gửi kèm promotionCode]

19 validate(promotionCode, pricing.subtotal)

**alt** [Mã khuyến mãi không hợp lệ hoặc không đủ điều kiện]

20 null

21 throw BadRequestException('Promotion code is invalid or not applicable.')

22 400 Bad Request

23 Hiển thị lỗi khuyến mãi

**[Mã khuyến mãi hợp lệ]**

24 { promotionId, discountAmount, ... }

25 prisma.$transaction(async (tx) => { ... })

26 tx.user.findUnique({ where: { id: userId } })

27 tx.product.findMany({ where: { id: { in: productIds }, isAvailable: true } })

28 tx.topping.findMany({ where: { id: { in: toppingIds }, isAvailable: true } })

**loop** [Duyệt từng sản phẩm: Kiểm soát tồn kho bằng Optimistic Locking]

29 tx.product.updateMany({
where: { id: productId, isAvailable: true, version: product.version, stock: { gte: quantity } },
data: { stock: { decrement: quantity }, version: { increment: 1 } }
})

**alt** [Tranh chấp đồng thời (version đổi) hoặc tồn kho không đủ (stock < quantity)]

30 result.count !== 1

31 throw BadRequestException('Insufficient stock or concurrent update for product ...')

32 400 Bad Request

33 Ném ApiError(400)

34 Hiển thị thông báo "Sản phẩm đã hết hàng hoặc có tranh chấp đặt hàng!"

**loop** [Duyệt từng topping: Trừ kho có điều kiện]

35 tx.topping.updateMany({
where: { id: toppingId, isAvailable: true, stock: { gte: quantity } },
data: { stock: { decrement: quantity } }
})

**alt** [Topping không đủ tồn kho]

36 result.count !== 1

37 throw BadRequestException('Insufficient stock for topping ...')

38 400 Bad Request

39 tx.order.create({
data: {
userId,
status: 'PENDING',
totalAmount,
discountAmount,
finalAmount,
items: { create: [...] },
statusLogs: { create: { fromStatus: null, toStatus: 'PENDING' } }
}
})

40 newOrder (OrderDetail)

41 newOrder

42 201 Created: {
success: true,
statusCode: 201,
data: OrderDetail,
timestamp
}

43 OrderDetail

44 setPendingOrder(order.id, currentSignature)

45 State updated

46 router.push(`/checkout/payment?orderId=${order.id}`)

Khách hàng (User)

Page: CheckoutPage (app/checkout/page.tsx)

Zustand: useCartStore (store/useCartStore.ts)

Client: order.service.ts (services/order.service.ts)

Guard: JwtAuthGuard (modules/auth/guards/jwt-auth.guard.ts)

Controller: OrdersController (modules/orders/orders.module.ts)

Service: OrdersService (modules/orders/orders.service.ts)

Service: PricingService (modules/pricing/pricing.service.ts)

Service: PromotionsService (modules/promotions/promotions.service.ts)

Database: PrismaService (prisma/prisma.service.ts)